

--PROJETO ACADÊMICO - PADRÕES DE PROJETO GOF E GRASP

--UNIVERSIDADE ESTADUAL PAULISTA

--Faculdade de Ciências e Tecnologia

--Campus de Presidente Prudente

--Bacharelado em Ciência da Computação

--INFORMAÇÕES GERAIS

Disciplina: Programação Orientada a Objetos II

Ano Letivo: 2025

Aluno: Alexandre Malavazi

Data de Implementação: Outubro/2025

Professor: [Milton Shimabukuro]

SUMÁRIO DE ATIVIDADES

Atividade 1 - Factory Method: Pizzaria

Atividade 2 - Adapter: Controles Playstation/Xbox

Atividade 3 - Abstract Factory: Lanchonete

Atividade 4 - GRASP: Low Coupling e High Cohesion

Atividade 5 - GRASP: Controller

ATIVIDADE 1 - FACTORY METHOD: PIZZARIA

OBJETIVO

Implementar o padrão Factory Method para simular uma pizzaria que fabrica diferentes sabores de pizza (Portuguesa, Calabresa, Marguerita, QuatroQueijos), demonstrando o conceito de Virtual Constructor.

PADRÃO APLICADO

Nome: Factory Method (Virtual Constructor)

Tipo: Padrão Criacional

Propósito: Definir uma interface para criar objetos, permitindo que subclasses decidam qual classe instanciar

ESTRUTURA DO PROJETO

Atividade1Alexandre_Malavazi/

```
├── src/
|   ├── Pizza.java
|   ├── PizzaPortuguesa_AlexandreMalavazi.java
|   ├── PizzaCalabresa_AlexandreMalavazi.java
|   ├── PizzaMarguerita_AlexandreMalavazi.java
|   ├── PizzaQuatroQueijos_AlexandreMalavazi.java
|   ├── FactoryPizza.java
|   └── Cliente.java
└── ComoCompilarExecutar.txt
```

COMPILAÇÃO E EXECUÇÃO

cd Atividade1AlexandreMalavazi/src

javac *.java

java Cliente

SAÍDA ESPERADA

=== MalavaziPizzas - FACTORY METHOD ===

Montando Pizza Portuguesa...

Montando Pizza Calabresa...

Montando Pizza Marguerita...

Montando Pizza Quatro Queijos...

ATIVIDADE 2 - ADAPTER: CONTROLES PLAYSTATION/XBOX

OBJETIVO:

Implementar o padrão Adapter para adaptar o uso de um controle de Playstation para um controle de Xbox, permitindo a compatibilidade entre interfaces incompatíveis.

PADRÃO APLICADO

Nome: Adapter (Wrapper)

Tipo: Padrão Estrutural

Propósito: Converter a interface de uma classe em outra interface esperada pelo cliente

ESTRUTURA DO PROJETO

Atividade2Alexandre_Malavazi/

```
|— src/
|   |— ControleXbox.java
|   |— ControlePlaystation.java
|   |— AdapterControle_AlexandreMalavazi.java
|   |— Cliente.java
|— ComoCompilarExecutar.txt
```

COMPILAÇÃO E EXECUÇÃO

cd Atividade2AlexandreMalavazi/src

javac *.java

java Cliente

SAÍDA ESPERADA

ATIVIDADE 2 - PADRÃO ADAPTER

=====

DIAGRAMA IMPLEMENTADO:

Cliente -> ControleXbox -> AdapterControle -> ControlePlaystation

1. Criando ControlePlaystation...

2. Criando AdapterControle...

TESTANDO ADAPTAÇÃO:

3. Cliente chamando acionarSensorXbox():

ADAPTADOR: Convertendo comando Xbox -> Playstation: Executando acao correspondente

=====

IMPLEMENTAÇÃO CONFORME DIAGRAMA CONCLUÍDA!

ATIVIDADE 3 - ABSTRACT FACTORY: LANCHONETE

OBJETIVO

Implementar o padrão Abstract Factory para criar famílias de objetos relacionados (lanches vegetarianos e não vegetarianos), permitindo a criação de produtos relacionados sem especificar classes concretas.

PADRÃO APLICADO

Nome: Abstract Factory

Tipo: Padrão Criacional

Propósito: Fornecer interface para criar famílias de objetos relacionados sem especificar classes concretas

ESTRUTURA DO PROJETO

Atividade3Alexandre_Malavazi/

```
├── src/
│   ├── AbstractFactory.java
│   ├── FactoryProdutor_AlexandreMalavazi.java
│   ├── FactoryLanche.java
│   ├── FactoryLancheVegetariano_AlexandreMalavazi.java
│   ├── FactoryLancheNaoVegetariano_AlexandreMalavazi.java
│   ├── Lanche.java
│   ├── BurgerTudo_AlexandreMalavazi.java
│   ├── BurgerSalada_AlexandreMalavazi.java
│   └── BurgerTudoVegetariano_AlexandreMalavazi.java
```

```
| └─ BurgerSaladaVegetariano_AlexandreMalavazi.java
|   └─ Cliente.java
└─ ComoCompilarExecutar.txt
```

COMPILAÇÃO E EXECUÇÃO

cd Atividade3Alexandre_Malavazi/src

javac *.java

java Cliente

SAÍDA ESPERADA

ATIVIDADE 3 - PADRÃO ABSTRACT FACTORY

=====

1. Obtendo fabrica de lanches NAO VEGETARIANOS...

2. Criando lanches nao vegetarianos:

Montando Burger Tudo: carne bovina, queijo, bacon, alface, tomate, maionese

Montando Burger Salada: carne bovina, alface, tomate, cebola, molho especial

3. Obtendo fabrica de lanches VEGETARIANOS...

4. Criando lanches vegetarianos:

Montando Burger Tudo Vegetariano: hamburguer de grão-de-bico, queijo vegano, alface, tomate, maionese vegana

Montando Burger Salada Vegetariano: hamburguer de lentilha, alface, tomate, cebola, molho vegano

ATIVIDADE 4 - GRASP: LOW COUPLING E HIGH COHESION

PROBLEMA PROPOSTO

Em um sistema de imobiliária, criar uma instância de Pagamento e associá-la a um Aluguel. Duas soluções foram analisadas:

Solução 1:

Imobiliária → registrarPagamento() → Aluguel → criar() → Pagamento

Solução 2:

Imobiliária → criar() → Pagamento → adicionarPagamento(p) → Aluguel

ANÁLISE GRASP

LOW COUPLING (BAIXO ACOPLAMENTO)

Solução 1: Alto acoplamento (Aluguel depende de Pagamento)

Solução 2: Baixo acoplamento (classes independentes)

Veredito: Solução 2 superior

HIGH COHESION (ALTA COESÃO)

Solução 1: Alta coesão (contexto natural no Aluguel)

Solução 2: Baixa coesão (lógica espalhada)

Veredito: Solução 1 superior

RECOMENDAÇÃO FINAL

Solução 2 é a mais apropriada devido à melhor arquitetura a longo prazo, priorizando baixo acoplamento que é mais crítico para sustentabilidade do sistema.

ATIVIDADE 5 - GRASP: CONTROLLER

DEFINIÇÃO DO PADRÃO

O Controller é um padrão GRASP que atribui a responsabilidade de tratar eventos do sistema a uma classe que representa a camada de controle entre a interface e a lógica de negócio.

ABORDAGENS ANALISADAS

CONTROLADOR DE FACHADA

Uma classe controla todos os casos de uso

Ponto único de entrada

Adequado para sistemas pequenos

Exemplo - Sistema Bancário:

[Cliente] → [BancoController] → [Conta, Transação, Cliente]

CONTROLADOR DE CASOS DE USO

Cada caso de uso tem seu controlador

Maior coesão e responsabilidade definida

Adequado para sistemas médios/grandes

Exemplo - Sistema Bancário:

[Cliente] → [SaqueController] → [Conta, Transação]

[Cliente] → [DepositoController] → [Conta, Transação]

RECOMENDAÇÃO

Para sistemas de médio a grande porte, a abordagem de Controlador de Casos de Uso é preferível devido à melhor coesão, facilidade de manutenção e aderência aos princípios SOLID.

METODOLOGIA DE DESENVOLVIMENTO

PRINCÍPIOS APLICADOS

Clean Code: Código limpo e legível

SOLID Principles: Aderência aos princípios de design

Design Patterns: Implementação correta dos padrões GoF

GRASP Patterns: Aplicação dos padrões de responsabilidade

CONCLUSÕES GERAIS

Este projeto demonstra a aplicação prática dos principais padrões de projeto GoF (Factory Method, Adapter, Abstract Factory) e padrões GRASP (Controller, Low Coupling, High Cohesion) no desenvolvimento de software orientado a objetos.

Desenvolvido por: Alexandre Malavazi

Data de Conclusão: Outubro/2025

Disciplina: Programação Orientada a Objetos II - 2025